

FunSearch (short for searching in the function space) is an artificial intelligence method developed by Google DeepMind for discovering computer programs that solve mathematical and algorithmic problems. It combines a large language model with an automated evaluator and an evolutionary search procedure, generating candidate programs, scoring them, and using high-performing programs to produce new candidates.

Google DeepMind announced FunSearch in 2023, and the associated paper was published in Nature. The system was applied to the cap set problem in extremal combinatorics and to the online bin packing problem, where it found new mathematical constructions and new packing heuristics.

Method

FunSearch frames a problem as a search over computer programs rather than as a direct search over individual solutions. The user provides a problem specification, an evaluation function, and an initial program or program skeleton. At each iteration, FunSearch samples existing programs from a database, favors higher-scoring programs, builds a prompt for a pretrained large language model, and asks the model to generate modified programs. The generated programs are then executed and scored by the evaluator.

The search process uses an island-based evolutionary method intended to maintain a diverse set of candidate programs and reduce the risk of becoming trapped in local optima. According to the original paper, one advantage of the approach is that FunSearch outputs programs that can sometimes be inspected, simplified, and interpreted by researchers, instead of only producing a final numerical answer or a large list of objects.

Algorithmic formulation

FunSearch can be described as a search over a space of program fragments, usually functions embedded in a fixed program skeleton. Let

F

$$\{\mathrm{\mathcal{F}}\}$$

be a space of candidate functions and let

S

:

F

$\rightarrow$

R

$$S:\mathcal{F}\rightarrow \mathbb{R}$$

be an evaluator score obtained by running a fixed solver that calls the candidate function.

Given an initial function

$$f$$

$$0$$

$$f_0$$

, FunSearch maintains a database

$$D$$

$$D$$

of evaluated, valid functions. It repeatedly samples high-scoring functions from

$$D$$

$$D$$

, uses them to construct prompts for a large language model, and asks the model to generate a new candidate function

$$f$$

$$,$$

$$f'$$

. The new function is executed inside the problem-specific program skeleton and scored by

the evaluator. If the generated function is valid, it is added back into

D

$$\{\displaystyle D\}$$

, allowing later prompts to build on stronger previous candidates.

In simplified form, the search loop can be written as:

D

←

{

(

f

0

,

S

(

f

0

)

)

}

$$D \leftarrow \{(f_0, S(f_0))\}$$

f

,

~

LLM

(
prompt

(

f

1

,

...

,

f

k

)

)

,

(

f

i

,

S

(

f

i

)

)

∈

D

$$\{\displaystyle f\sim \operatorname{LLM}(\operatorname{prompt}(f_{\{1\}},\ldots ,f_{\{k\}})),\quad (f_{\{i\}},S(f_{\{i\}}))\in D\}$$

if

f

,

is valid, then

D

←

D

U

{

(

f

,

,

S

(

f

,

)

)

}

.

$$\{\text{if } f \text{ is valid, then } D \leftarrow D \cup \{(f, S(f))\}.$$

The idealized objective is to find a candidate function with a high evaluator score,

f

*

∈

*

a

r

g

m

a

x

f

€

F

S

(

f

)

,

$$f^* \in \operatorname{arg\,max}_{f \in \mathcal{F}} S(f),$$

although in practice FunSearch returns the best valid function discovered during the search. The original implementation uses an island-based evolutionary process to preserve diversity among candidate functions while favoring higher-scoring programs.

Applications

Cap set problem

FunSearch was first demonstrated on the cap set problem, a problem in additive combinatorics concerning the largest possible subset of

$\mathbb{Z}$

3

n

$$\{\displaystyle \mathbb{Z}_{_3}^n\}$$

with no three points in a line. In dimension 8, FunSearch found a cap set of size 512, improving on previously known constructions. The paper also reported improved lower bounds for the cap set capacity by using FunSearch to discover constructions related to admissible sets.

Google DeepMind described the result as an example of using large language models to generate verifiable new knowledge in mathematics. A Nature news article reported that the system improved on human efforts for a combinatorics problem related to the card game Set.

Online bin packing

FunSearch was also applied to the online bin packing problem, where items must be assigned to bins as they arrive. In this setting, FunSearch evolved programmatic heuristics that decide which bin should receive a new item. The original paper reported that the discovered heuristics outperformed the common first-fit and best-fit baselines on simulated data and OR-Library benchmark instances.

Software

Google DeepMind released the FunSearch software in a public GitHub repository accompanying the FunSearch paper. The repository includes examples and data for cap sets, admissible sets, online bin packing, independent sets in strong products of cycle graphs, corner-free sets, and related experiments. It also includes a single-threaded implementation of the FunSearch pipeline, although the repository notes that it does not include the language models, execution sandbox, or distributed infrastructure used in the original experiments. The

repository states that its software is licensed under the Apache License 2.0, while other materials are licensed under the Creative Commons Attribution 4.0 International License.

Reception

In a Nature News & Views article, Jean-Baptiste Mouret described FunSearch as connecting genetic programming with large language models, writing that the work showed how language models could help computer programs evolve. The article characterized the system as a proof of concept for AI-assisted mathematical discovery.

See also

[AlphaDev](#)

[AlphaEvolve](#)

[AlphaGeometry](#)

[AlphaTensor](#)

[Automated theorem proving](#)

[Evolutionary algorithm](#)

[List of artificial intelligence algorithms](#)

[List of unsolved problems in mathematics](#)

[Program synthesis](#)

References

[External links](#)

[Official website](#)

[funsearch on GitHub](#)